

Build Your Own Compiler

A Practical Walkthrough on Linux
Using Python • Targeting x86-64 Assembly (or C)

This guide takes you from zero to a working compiler that turns a tiny C-like language into real machine code. We follow the proven incremental approach: start with the smallest possible program and grow one feature at a time.

What you will build

- A complete compiler written in Python
- Lexer → Parser → AST → Code Generator pipeline
- Output: real x86-64 Linux assembly (or C as an easier alternative)
- Ability to expand it into a much richer language

Designed for beginners who know Python and want to understand how compilers really work.

Contents

1. Introduction & Philosophy
2. Prerequisites & Setup on Linux
3. The Tiny Language We Will Compile
4. Stage 1 — Compile Integer Constants (Full Working Code)
5. Understanding the Pipeline
6. How to Expand the Compiler
7. Alternative: Compile to C Instead of Assembly
8. Recommended Learning Path & Resources

1. Introduction & Philosophy

Writing a compiler is one of the most educational projects a programmer can undertake. You will deeply understand abstract syntax trees, how high-level constructs map to machine instructions, calling conventions, and the entire journey from source text to executable.

The Incremental Approach (Critical)

Do **not** try to implement a full language at once. The modern, successful way (popularized by Abdulaziz Ghuloum and used by Nora Sandler in her excellent C compiler series) is:

1. Write a compiler that only handles the absolute minimum language.
2. Make sure it produces a **working executable**.
3. Add one new language feature.
4. Repeat.

Your first compiler will only understand programs of the form:

```
int main() { return 42; }
```

That is enough. Once that works end-to-end, everything else becomes much easier.

This approach keeps motivation high because you always have a working program, and each step is small enough to debug.

2. Prerequisites & Setup on Linux

You need very little to begin:

- A Linux machine (or WSL2 / any modern Unix)
- Python 3.8 or newer
- gcc (to assemble and link the generated assembly)
- A text editor

Install the essentials if needed:

Terminal

```
sudo apt update
sudo apt install build-essential python3    # Debian/Ubuntu
# or
sudo dnf install gcc python3                # Fedora
```

Create the project directory:

Project setup

```
mkdir -p ~/mycompiler/tests
cd ~/mycompiler
touch compiler.py
```

3. The Tiny Language We Will Compile

For Stage 1 we support only this grammar (written in a simple BNF style):

Stage 1 Grammar

```
<program>    ::= "int" "main" "(" ")" "{" <statement> "}"  
<statement>  ::= "return" <expr> ";"  
<expr>       ::= <integer>  
<integer>    ::= [0-9]+
```

In other words, the only valid programs look like:

```
int main() {  
    return 42;  
}
```

Later we will add unary and binary operators, variables, if-statements, loops, and functions. But for now this is perfect.

4. Stage 1 — Compile Integer Constants

Below is a complete, self-contained Python compiler for Stage 1. Copy it into `compiler.py`.

We will build the compiler in four clear pieces. Create a file called `compiler.py` and add the sections one by one.

Part A — Tokens and AST Nodes

Data classes

```
#!/usr/bin/env python3
"""Stage 1 Compiler: int main() { return <number>; }"""

import sys
from dataclasses import dataclass
from typing import List

@dataclass
class Token:
    type: str
    value: str
    line: int = 1

@dataclass
class Program:
    function: "Function"

@dataclass
class Function:
    name: str
    body: "Return"

@dataclass
class Return:
    expr: "Constant"

@dataclass
class Constant:
    value: int
```

Part B — The Lexer

The lexer turns source text into a list of tokens. It is deliberately simple.

Lexer

```

def tokenize(source: str) -> List[Token]:
    tokens = []
    i = 0
    line = 1
    keywords = {"int", "main", "return"}

    while i < len(source):
        c = source[i]

        if c in " \t\r":
            i += 1
            continue
        if c == "\n":
            line += 1
            i += 1
            continue

        if c in "(){};":
            tokens.append(Token(c, c, line))
            i += 1
            continue

        if c.isdigit():
            start = i
            while i < len(source) and source[i].isdigit():
                i += 1
            tokens.append(Token("NUMBER", source[start:i], line))
            continue

        if c.isalpha() or c == "_":
            start = i
            while i < len(source) and (source[i].isalnum() or source[i] == "_"):
                i += 1
            word = source[start:i]
            tok_type = word if word in keywords else "IDENT"
            tokens.append(Token(tok_type, word, line))
            continue

        raise SyntaxError(f"Unexpected character '{c}' on line {line}")

    tokens.append(Token("EOF", "", line))
    return tokens

```

Part C — The Parser (Recursive Descent)

A recursive-descent parser is the easiest to write and understand. It directly mirrors the grammar.

Parser

```

class Parser:
    def __init__(self, tokens: List[Token]):
        self.tokens = tokens
        self.pos = 0

    def current(self) -> Token:
        return self.tokens[self.pos]

    def eat(self, expected_type: str) -> Token:
        tok = self.current()
        if tok.type != expected_type:
            raise SyntaxError(
                f"Expected {expected_type}, got {tok.type} "
                f"('{tok.value}') on line {tok.line}"
            )
        self.pos += 1
        return tok

    def parse(self) -> Program:
        self.eat("int")
        self.eat("main")
        self.eat("(")
        self.eat(")")
        self.eat("{")
        self.eat("return")
        expr = self.parse_expr()
        self.eat(";")
        self.eat("}")
        self.eat("EOF")
        return Program(Function("main", Return(expr)))

    def parse_expr(self) -> Constant:
        tok = self.eat("NUMBER")
        return Constant(int(tok.value))

```

Part D — Code Generator & Driver

The code generator walks the AST and emits x86-64 AT&T; assembly. On Linux the return value of main goes in the %rax register.

Code generator + main

```

def generate(program: Program) -> str:
    value = program.function.body.expr.value
    lines = [
        "    .globl main",
        "main:",
        f"    movq ${value}, %rax",
        "    ret",
        ""
    ]
    return "\n".join(lines)

def compile_file(source_path: str, output_path: str = None):
    with open(source_path) as f:
        source = f.read()

    tokens = tokenize(source)
    program = Parser(tokens).parse()
    asm = generate(program)

    if output_path is None:
        output_path = source_path.rsplit(".", 1)[0] + ".s"

    with open(output_path, "w") as f:
        f.write(asm)
    print(f"Generated: {output_path}")
    return output_path

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python compiler.py <file.c> [output.s]")
        sys.exit(1)
    src = sys.argv[1]
    out = sys.argv[2] if len(sys.argv) > 2 else None
    compile_file(src, out)

```

How to Compile and Run a Program

Create a test file and run the full pipeline:

Full pipeline on Linux

```
# 1. Write a tiny program
echo 'int main() { return 42; }' > test.c

# 2. Run your compiler (produces test.s)
python3 compiler.py test.c

# 3. Assemble + link with gcc
gcc test.s -o test

# 4. Run it and check the exit code
./test
echo $?
```

You should see 42 printed. Congratulations — you just wrote a real compiler!

The assembly that was generated looks like this:

Generated test.s

```
.globl main
main:
    movq $42, %rax
    ret
```

On the System V AMD64 ABI used by Linux, the return value of a function lives in the %rax register. The ret instruction returns to the caller (the C runtime).

5. Understanding the Pipeline

Lexer (Tokenizer)

The lexer reads the raw source text character by character and produces a list of tokens. It ignores whitespace and comments (we have none yet) and recognizes keywords, numbers, and punctuation. A good lexer never tries to understand structure — that is the parser's job.

Parser + Abstract Syntax Tree (AST)

We use a hand-written recursive-descent parser. It is the simplest and most educational technique. The parser consumes tokens and builds an AST — a tree data structure that represents the meaning of the program. In Stage 1 the tree is very small:

AST for return 42

```
Program
  ■■■ Function(name="main")
    ■■■ Return
      ■■■ Constant(value=42)
```

Code Generator

The code generator walks the AST and emits the corresponding assembly instructions. Because our language is still tiny, the generator is almost trivial. As the language grows, this is where most of the complexity will live (register allocation, stack frames, calling conventions, etc.).

6. How to Expand the Compiler

Add features one at a time. Here is a recommended order (the same order used by Nora Sandler's excellent series and Ghuloum's paper):

1. Unary operators ($-$, $!$, $\sim$)

Add tokens for the operators. Extend the expression grammar and AST. In the code generator emit the appropriate instructions (neg, not, etc.).

2. Binary arithmetic ($+$, $-$, $*$, $/$)

This is the first place where you need to think about temporary values and registers (or the stack). A simple approach is to evaluate the left side, push it, evaluate the right side, then pop and perform the operation.

3. Local variables

Introduce a symbol table (a simple dictionary is enough at first). Allocate space on the stack for each variable and emit the correct offsets.

4. Assignment and expressions with variables

5. if / else (conditional jumps)

6. Loops (while / for)

7. Functions (calling convention, stack frames, arguments)

At every step keep the test suite growing. Write small .c files that exercise the new feature and verify the exit code (or printed output once you add I/O).

Tip: After each feature, you can still compile the old programs. Regression testing is essential.

7. Alternative: Compile to C Instead of Assembly

If assembly feels intimidating at first, you can change the code generator to emit C instead of assembly. Then just run gcc on the generated C file. This is a perfectly valid and educational intermediate step.

Replace the generate function with something like:

Code generator that emits C

```
def generate_c(program: Program) -> str:
    value = program.function.body.expr.value
    return f'''
#include <stdio.h>
int main(void) {{
    return {value};
}}
'''
```

Later you can replace this backend with a real assembly (or LLVM IR) backend without changing the frontend.

8. Recommended Learning Path & Resources

Immediate next step

Finish Stage 1, then add unary negation and binary addition. You will learn a surprising amount from just those two features.

Best Practical Resources

- **Nora Sandler – Writing a C Compiler** (free blog series + published book)

The gold standard for the incremental approach targeting real assembly.

- **Abdulaziz Ghuloum – An Incremental Approach to Compiler Construction** (paper)

The original inspiration for the “start tiny” method.

- **Jack Crenshaw – Let’s Build a Compiler** (classic free series)

Extremely readable introduction to recursive-descent parsing.

- **Crafting Interpreters** by Robert Nystrom

Excellent if you want to build an interpreter / bytecode VM first (highly recommended before a full native compiler).

Theory (read later)

- Compilers: Principles, Techniques, and Tools (the “Dragon Book”)
- Engineering a Compiler (Cooper & Torczon)

You now have everything you need to start. Open a terminal, create the project, paste the Stage 1 code, and get a program that returns 42 working today. Then grow it. That is how real compilers are built.

Happy compiling.